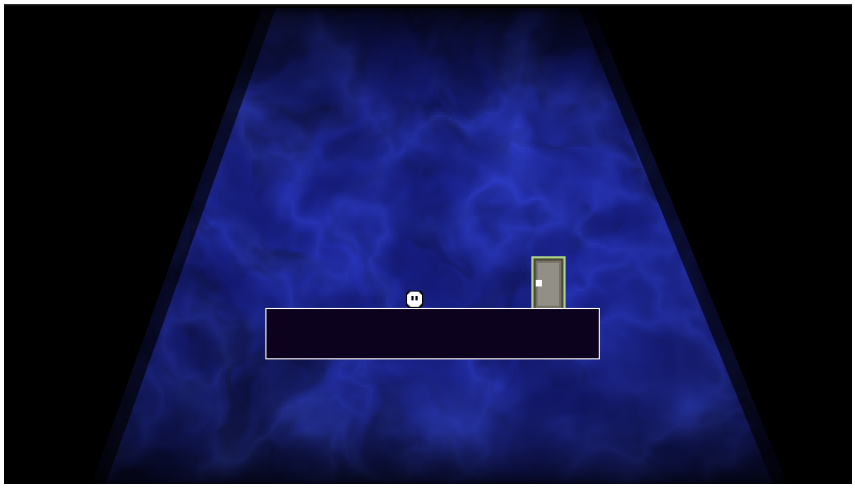
Light Bender

Game Design Document



Built with Godot 4.6 · GDScript · PC

Game Name:

Light Bender

Genre:

2D Puzzle Platformer — Single-player

Game Elements:

- **Light manipulation** — bend, redirect, carry, and toggle light sources to reshape the traversable world.
- **Precision platforming** — jump, dash, and wall-slide through dark, physics-driven rooms.
- **Object interaction** — absorb batteries and flashlights, carry them to new positions, drop them as physical puzzle pieces.
- **Switch puzzles** — flip switches to turn light zones on or off, opening or closing paths and platforms.
- **Exploration** — progress through Chapter 1's interconnected rooms via a visual level selector.

Player:

Single-player only. One player controls the character at a time; no local or online multiplayer is planned.

TECHNICAL SPECS

Technical Form:

2D — flat graphics. Light Bender is a fully 2D game rendered with Godot's 2D pipeline. The darkness overlay, light zones, and background shaders are implemented with Godot CanvasModulate, Polygon2D masks, and GLSL shaders. The target viewport is 1920×1080.

View:

Side-scrolling orthographic 2D view. The camera follows the player character within each room. No perspective projection or Z-depth is used.

Platform:

PC (Windows, macOS, Linux). The GL Compatibility renderer is used so the game can run on a wide range of hardware without requiring Vulkan.

Language:

GDScript — Godot's built-in scripting language. All gameplay logic, UI, level management, audio, and shader control is written in GDScript. No external scripting languages or compiled plugins are used.

Device:

PC with keyboard. The full control scheme uses `A`/`D`/arrow keys for movement, `Space`/`W`/`Up` for jump, `E` to interact, `F` to pick up/drop objects, `R` to rotate held flashlight, and `Esc` to pause. Controller support is not implemented in the current build.

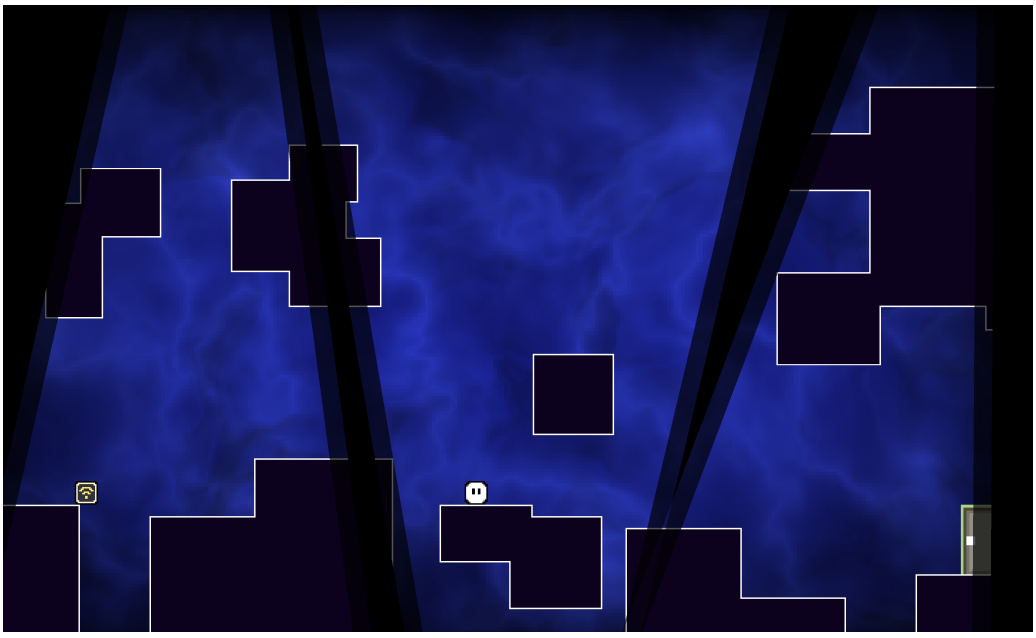
GAME PLAY

You open Light Bender and land on the Chapter 1 level selector — a graph of interconnected rooms that tracks which levels you have cleared. Selecting a room fades in through a circular transition and drops you in front of an entrance door.

The moment you step inside, the world is mostly dark. Only the lit polygons cut through the darkness overlay are real: platforms you can stand on, doors you can open, switches you can flip. Step outside the light and the floor dissolves beneath you. This single rule — *if it is not in the light, it does not exist* — is the entire game.

Each room asks you to solve the shape of the lit world. You may find a battery resting in a beam of light, slowly charging. Press `F` and the battery becomes part of you: a portable island of safety glowing in your hands. Carry it into the dark corner you could not reach, set it down, and a new platform comes to life beneath your feet.

A flashlight box works differently. You can pick it up, rotate it with `R` while carrying it, and angle its narrow beam into a room alcove that was previously unlit. That alcove might hide a switch. Pressing `E` on the switch toggles a remote light section, revealing a bridge you could not see before. Cross it. Find the exit door. Press `E` again and the circular transition closes around you — level complete.



Chapter 1 — a larger room with multiple lit zones and platforming routes.

Game Play Outline

1. **Opening the game application**
   The game launches directly into the Level Selector scene. A procedural shader background (blue, fluid shapes) and pixel-art UI greet the player.

2. **Game options**
   From the Level Selector the player can open the pause/settings menu (or press `Esc` in-level). A sliding panel offers independent Music and SFX volume sliders. Settings are persisted through `LevelManager`.

3. **Story synopsis**
   Light Bender has no written dialogue. The world communicates its rules through environmental design: darkness is hostile, light is safe, and light is moveable.

4. **Modes**
   One mode: Story/Puzzle. All eleven Chapter 1 rooms are played in the order the player chooses from the selector graph, with earlier rooms needing to be cleared to unlock later ones.

5. **Game levels**
   Chapter 1 contains eleven rooms of increasing complexity. Each room is a self-contained puzzle built around one or more light mechanic concepts. Future chapters (2 and 3) introduce new shaders and will unlock the remaining player abilities (dash, double jump, wall movement, slow fall).

6. **Winning**
   The player wins a room by reaching and interacting with the exit door while the door is in a lit zone. The level selector updates to mark the room completed and unlocks the next room.

7. **Losing**
   The player dies by touching a lit hazard. There is no health bar; death is instant. The circular transition triggers, and the player respawns at the last checkpoint door. Progress is not lost.

8. **End**
   Completing all Chapter 1 rooms marks the chapter finished on the selector. The game is currently scoped to three chapters; Chapters 2 and 3 are in design/pre-production.

9. **Why is all this fun?**
   The core satisfaction is *creative path-finding*: the route does not pre-exist, the player builds it by shaping light. Carrying a battery into darkness and watching a platform materialise beneath your feet is immediately legible and endlessly satisfying. The movement controller adds a second layer of fun that rewards mastery even when a puzzle is simple.

Key Features

Feature	Why it is attractive
Light-driven world	Every room is a spatial puzzle where light defines reality; no two rooms feel the same.
Movement-first controller	Coyote time, jump buffering, dash, wall movement, double jump, slow fall, and squash/stretch make every traversal feel responsive and expressive.
Portable light objects	Batteries and flashlights can be carried and repositioned; objects become physical platforms when dropped — emergent puzzle solutions arise naturally.
Switch-based routing	Switches let the player toggle whole sections of the world on or off, creating before/after transformations of the same room.
Chapter progression	A visual level selector with locked/unlocked/completed states gives a sense of journey and reward.
Procedural backgrounds	Each chapter has a unique generative shader background that makes the world feel alive without expensive art assets.
Circular transitions	Deaths, level changes, and menu navigation all share the same smooth ring transition, giving the game a coherent, polished rhythm.
Future-ready abilities	Dash, double jump, wall movement, and slow fall are fully implemented but gated, ensuring later chapters can introduce them as meaningful unlocks.

DESIGN DOCUMENT

Design Guidelines

- **One rule governs everything**: if an object is not in the light, it does not reliably exist for gameplay purposes. Every mechanic must be explainable by this rule.
- **Movement feels good first**: the player spends the whole game running and jumping; the controller must feel fair, fluid, and forgiving before any puzzle is built around it.
- **Light as game state, not decoration**: light zones are not visual polish; they are binary gameplay switches. A platform in darkness is not a platform.
- **No tutorial text**: all mechanics are taught through level design. A new object is introduced in a safe, obviously lit context before it is used as a puzzle element.
- **Pixel aesthetic consistency**: all UI is pixel-art styled. No anti-aliased fonts or photo assets. The visual language stays flat, readable, and cohesive across menus and levels.
- **Reusable systems**: new rooms use existing features. One-off hacks go into `features/` as reusable scenes before being placed in a level.

Game Design Definitions

Term	Definition
Light Zone	A polygon-shaped region that cuts through the darkness overlay. Objects inside are "lit"; objects outside are "dark".
Dark	The default state of the world. Platforms, hazards, switches, and doors in the dark are either non-solid, inactive, or safe (depending on type).
Lit	The active state. Platforms are solid, hazards are lethal, switches are interactable, and doors are openable.
LightReactive	A base behaviour any object can carry. It listens to `LightReceiver` signals and enables/disables its gameplay properties accordingly.
Absorb (Pick up)	The player presses `F` near an absorbable object. The object attaches to the player, retains its behaviour (light emission, collision), and can be repositioned and dropped.
Checkpoint	A door marked as a checkpoint. Touching it saves the respawn position for the current room.
Win Condition	The player interacts (`E`) with an exit door that is inside a lit zone.
Lose Condition	The player character touches a hazard object that is in a lit zone.
Transition	A circular wipe effect that plays on death, level entry, level exit, and menu navigation. Implemented by `CircleTransition`.

Game Flowchart

High-level flow:

Launch → **Level Selector** → **Select Room** → [**CircleTransition in**] → **Room Gameplay Loop**

Manipulate light → *Platform / interact* → *Reach exit door*

→ **Win:** [**CircleTransition out**] → **Level Selector (room marked complete)**

→ **Lose (hazard):** [**CircleTransition**] → **Respawn at checkpoint**

→ **Pause Menu** → *Resume / Audio settings / Return to selector*

Light system data flow:

`DarkManager` (overlay) → `LightZone` (polygon cutout) → `LightReceiver` (overlap signal)

→ `LightReactive` (enable/disable behaviour) → *Platforms, Hazards, Doors, Switches, Batteries, Flashlights*

Player interaction flow:

Input → `PlayerMovement` (physics, coyote, buffering) + `PlayerAbilities` (dash, wall, double jump)

→ `PlayerPickupController` (absorb / carry / drop)

→ *Object placed in world* → resumes LightReactive behaviour

Player Definition

The player controls a small humanoid character who is always safe from darkness — the character itself is not light-reactive and can walk into dark zones freely. The danger comes from the world: dark platforms disappear, lit hazards kill. The player's job is not to survive darkness but to *reshape the light* until the world is traversable.

The character has a squash-and-stretch sprite that responds to jumps, landings, and dashes, giving physical weight to movement.

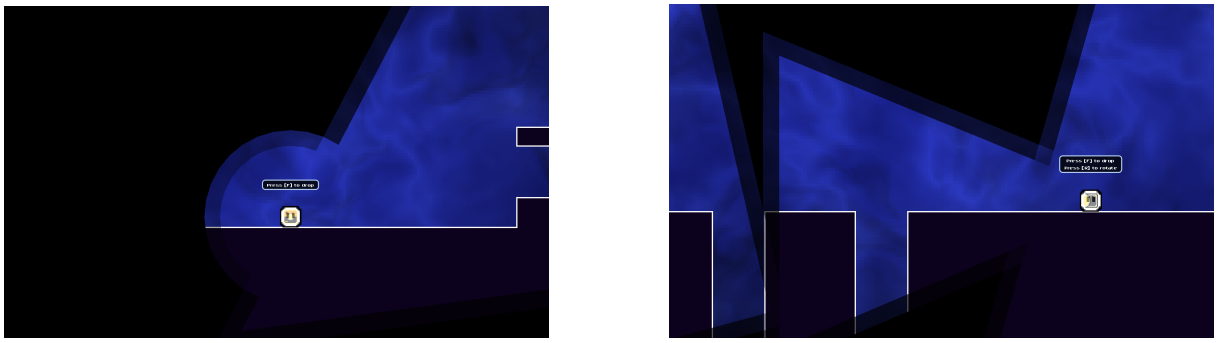
- **Health**: Binary. The player is alive or dead; no HP bar. One touch of a lit hazard kills instantly.
- **Abilities**: Move, jump (variable height), coyote jump, buffered jump, fast fall, dash (Chapter 2+), wall slide/jump (Chapter 2+), double jump (Chapter 3+), slow fall (Chapter 3+).
- **Actions**: Interact (`E`), pick up/drop (`F`), rotate held object (`R`), pause (`Esc`).

Player Properties

Property	Description	Feedback
Alive / Dead	Binary health state.	On death: CircleTransition closes; SFX plays; respawn at checkpoint.
Grounded	Whether the player is on a solid surface.	Affects available jumps, coyote timer, squash/stretch on landing.
Carrying object	Whether an absorbable object is held.	Object follows player position; its light/collision stays active; visual prompt hidden.
Coyote time	Short window after leaving a ledge when jumping is still allowed.	Player can jump slightly after walking off a platform; prevents unfair misses.
Jump buffer	Jump input stored for a few frames before landing.	Early jump input registers on landing; movement feels forgiving.
Dash available	Whether the dash action can be used (Chapter 2+).	Dash trail appears during dash; short cooldown prevents spam.

Player Rewards (power-ups and pick-ups)

Object	**Effect on player**
Battery (charged)	Carried light source; creates a safe lit zone around the player in otherwise dark areas; can be placed as a platform.
Flashlight	Directional beam the player can aim; illuminates specific targets; can be carried and rotated (`R`) to solve directional puzzles.
Checkpoint door	Saves respawn position; reduces punishment on death; no visual reward but critical quality-of-life benefit.
Exit door (reached)	Completes the level; triggers CircleTransition; marks room complete in LevelManager; unlocks next room.



Left: Battery carried as a portable light source. Right: Flashlight rotation puzzle.

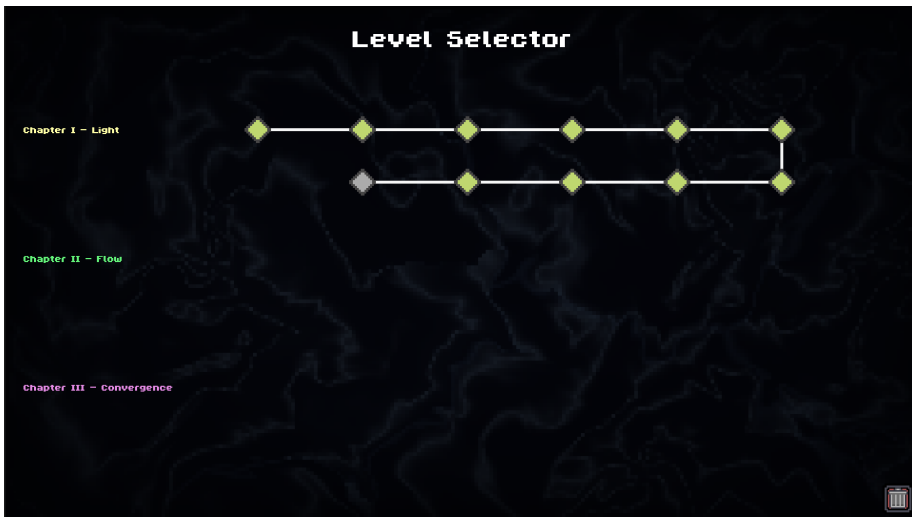
User Interface (UI)

Controls

Action	Key
Move left / right	`A` / `D` or arrow keys
Jump	`Space`, `W`, or `Up`
Fast fall	`S` or `Down`
Dash (Chapter 2+)	`K` or `Shift`
Interact / enter door / toggle switch	`E`
Pick up / drop object	`F`
Rotate held flashlight	`R`
Pause	`Esc`
Debug respawn	`P`
Debug restart level	`O`

Level Selector

The Level Selector is the main hub. It reads from `LevelManager.LEVELS` and builds a chapter graph automatically. Each node shows the room name and a status icon: locked (grey), unlocked (white), or completed (coloured). Clicking an unlocked node launches the room.



Chapter 1 level selector with unlocked and completed rooms.

Pause / Settings Menu

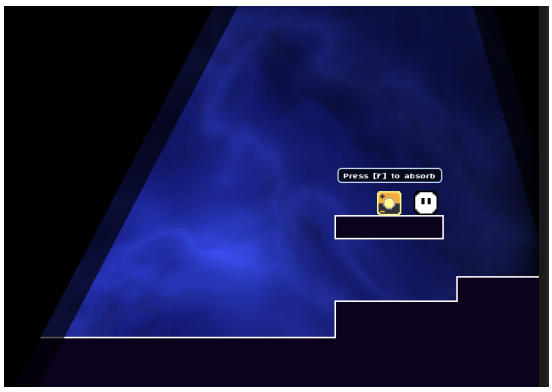
The pause menu overlays the current scene without unloading it. It offers:

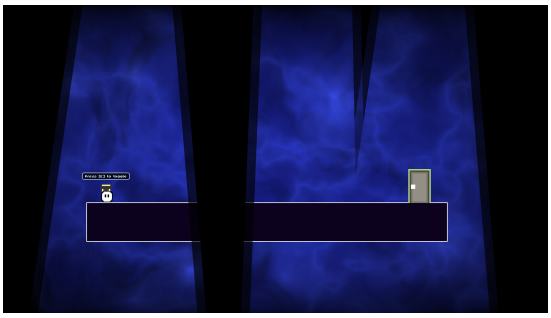
- Resume button
- Music volume slider (persisted to `LevelManager`)
- SFX volume slider (persisted to `LevelManager`)
- Return to Level Selector button

Pause menu with audio sliders.

In-level UI

Minimal HUD by design. The world communicates state through light and sprite feedback, not icons or bars. Interaction prompts (`E` / `F`) appear above objects when the player is in range. No health bar, no score display, no timer.

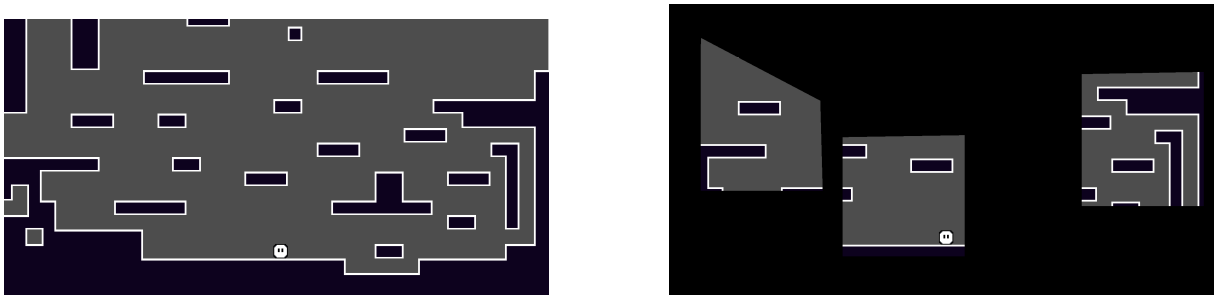




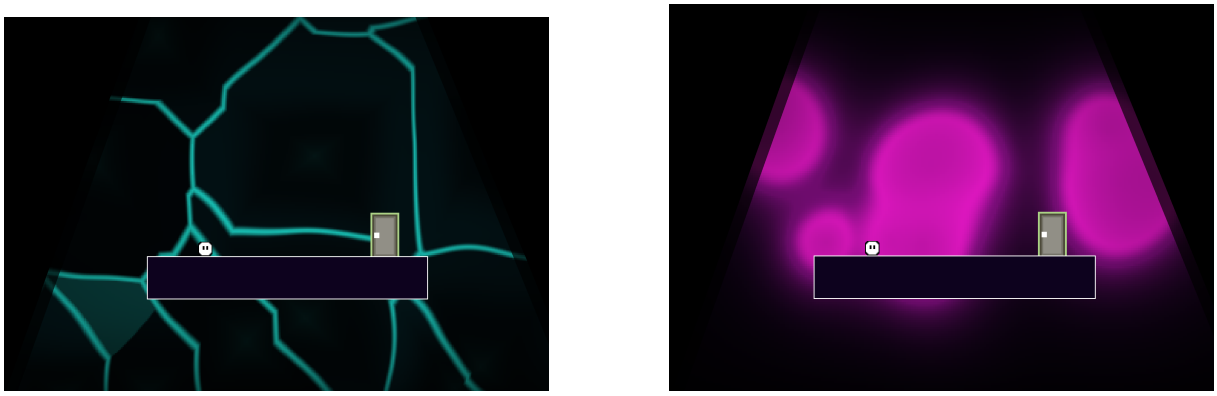
Left: Interaction prompt above a battery. Right: Switch-and-door puzzle layout.

VISUAL DEVELOPMENT REFERENCE

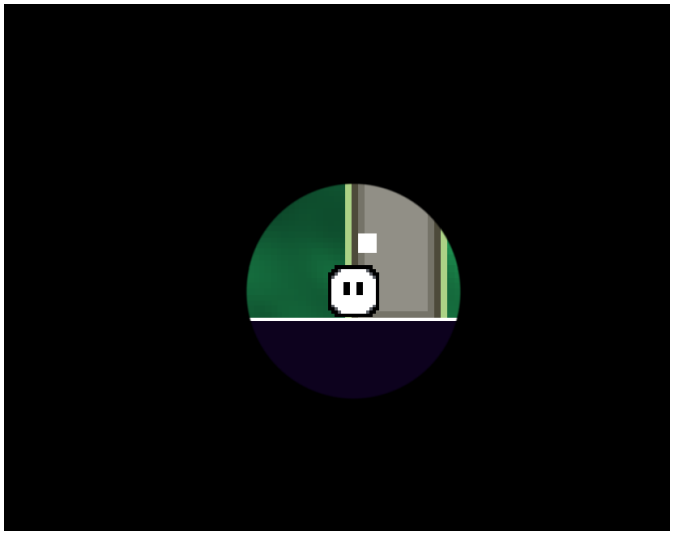
The following screenshots document the visual evolution of Light Bender from prototype to Chapter 1 release.



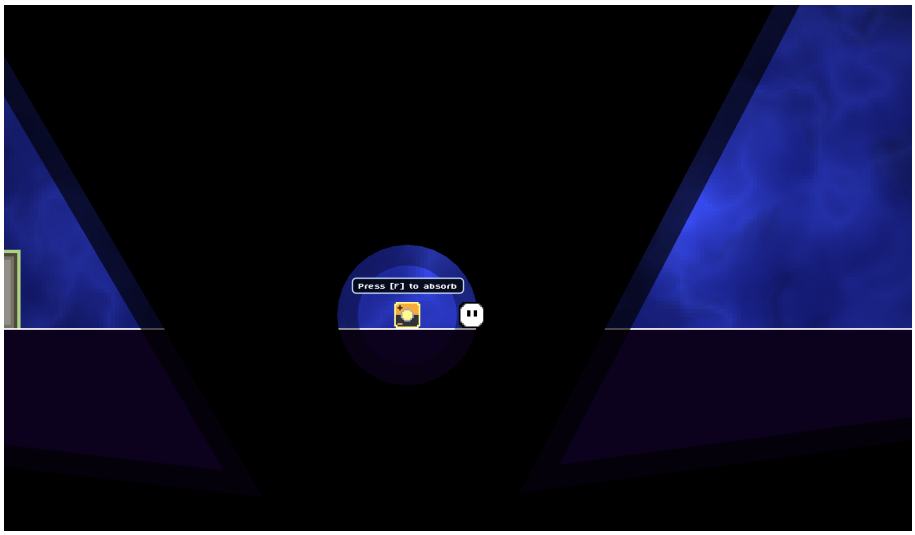
Left: Early player movement prototype. Right: First light mechanic prototype.



Left: Chapter 2 procedural background shader. Right: Chapter 3 background shader.



Circular transition ring used for deaths, level entry/exit, and menu navigation.



Battery light illuminating an otherwise completely dark room section.